

目录

1 引言

- 1.1 分布式哈希表的概念
- 1.2 DHT 的主要特征
- 1.3 DHT 的应用场景

2 DHT 的基本原理

- 2.1 哈希空间与节点映射
- 2.2 数据存储与查找
- 2.3 节点加入与退出
- 2.4 容错与负载均衡

3 Chord 协议详解

- 3.1 环形拓扑结构
- 3.2 一致性哈希
- 3.3 路由表与指针维护
- 3.4 查询机制
- 3.5 节点动态管理

4 Kademlia 协议详解

- 4.1 XOR 度量空间
- 4.2 k-bucket 路由表
- 4.3 节点查找算法
- 4.4 并行查找优化
- 4.5 数据复制与缓存

5 Chord 与 Kademlia 的比较

- 5.1 路由效率对比
- 5.2 可扩展性分析
- 5.3 容错性能对比
- 5.4 实现复杂度

6 总结与展望

1 引言

随着互联网规模的指数级增长和对去中心化、抗审查网络的需求日益提升，传统的集中式数据存储和检索方案面临着可扩展性、单点故障和性能瓶颈等诸多挑战。分布式哈希表（Distributed Hash Table, DHT）技术应运而生，它提供了一种在无需中央协调机构的情况下，将数据的存储和路由责任分散到数千个乃至数百万个对等节点上的高效解决方案。

1.1 分布式哈希表的概念

分布式哈希表是一种基础性的分布式系统架构，其核心功能是实现键（Keys）到值（Values）的映射，本质上是将经典的哈希表功能在去中心化网络环境中实现^[1]。与传统数据库或服务器-客户端架构不同，DHT 利用网络中所有对等节点（Peer）的计算和存储能力，共同维护一个巨大的键值对集合。

在现代去中心化网络（如星际文件系统 IPFS）中，DHT 扮演着关键的内容路由系统组件的角色，其作用类似于一个目录与导航系统的交叉体^[1]。它的主要职责是将用户请求的内容或资源标识符，精确地映射到当前网络中存储该内容的对等节点地址^[1]。因此，DHT 不仅是数据存储的定位器，更是去中心化网络进行高效寻址的基础设施。

1.2 DHT 的主要特征

为适应大规模、动态变化的对等网络环境，DHT 系统在设计时通常高度强调以下三个核心特性^[2]：

特性	描述	设计目标
自主性与去中心化	系统中的节点以集体形式运作，不依赖任何中心服务器或权威机构进行协调。	保证系统的抗审查性、高可用性和鲁棒性 ^[2] 。
容错性	系统必须具备弹性，即使面对节点持续加入、离开或随机故障（即高流失率，Churn），仍能保持功能完整和数据可靠 ^[2] 。	通过冗余存储和动态维护机制实现网络稳定性。
可扩展性	系统必须能够高效地扩展至数千甚至数百万个节点，且查询效率不应随网络规模线性下降 ^[2] 。	路由查找的平均时间复杂度应达到网络规模的对数级别 $O(\log N)$ ^[1] 。

高效的 $O(\log N)$ 路由复杂度是 DHT 成功的关键。这意味着即使网络规模 N 呈指数级增长，定位数据所需的平均步骤也仅呈线性增长，确保了大规模应用的可行性。

1.3 DHT 的应用场景

DHT 的核心功能是实现高效的地址解析和内容寻址。在实际应用中，DHT 被用于映射多种类型的关键记录，从而支撑去中心化网络的正常运行^[3]：

- 提供者记录 (Provider records)**：这是内容路由的基础。DHT 将数据标识符（通常是内容的哈希值，如 `multihash`）映射到已宣布拥有并愿意提供该内容的对等节点地址。在 IPFS 中，系统通过查找这些记录来定位所需内容^[1]。
- 去中心化命名服务记录 (IPNS records)**：DHT 支撑着去中心化的命名系统。它将 IPNS 密钥（例如公钥的哈希）映射到一个经过签名和版本控制的指针，该指针指向特定的路径（如内容地址 `/ipfs/bafyxyz...`）^[1]。
- 对等节点记录 (Peer records)**：用于网络连通性维护和地址解析。DHT 将对等节点 ID (PeerID) 映射到一组可以联系到该节点的网络地址（`multiaddress`）。当系统知道哪个节点拥有内容，但需要知道如何连接到它时，会查询这些记录^[1]。

综上所述，DHT 技术通过其独特的去中心化、容错和高可扩展性，已经成为构建新一代互联网基础设施——特别是大规模 P2P 文件共享、内容分发网络和区块链辅助存储——不可或缺的核心技术。本文后续将深入探讨两种最具代表性的结构化 DHT 协议：Chord 和 Kademlia。

2 DHT 的基本原理

分布式哈希表（DHT）的核心原理在于将传统的哈希表功能扩展到去中心化的对等网络中，通过一个统一的机制管理键空间、节点映射、数据定位以及网络的动态变化。

2.1 哈希空间与节点映射

DHT 系统的基础在于构建一个统一的地址空间（Address space），所有节点和数据键都被映射到这个空间中。

1. **统一 ID 空间：** 无论是网络中的对等节点 ID 还是需要存储的数据键，都通过一个一致的哈希函数（例如 SHA256）映射为一个具有相同格式和长度的数值标识符。例如，在 IPFS 对 Kademlia 的实现中，地址空间是 0 到 $2^{256} - 1$ 的所有数字，即一个 256 位的哈希空间^[1]。
2. **节点度量与排序：** DHT 采用特定的度量（Metric）或距离函数来在地址空间中对节点进行排序。
 - 在 Chord 协议中，度量是基于数值顺序和模运算，将节点排列成一个逻辑环^[4]。
 - 在 Kademlia 协议中，度量是通过 XOR（异或）运算来计算 ID 之间的距离^[5]。

通过这种度量，整个对等网络可以被逻辑上可视化为沿一条线或一个环排序的结构。

2.2 数据存储与查找

数据在 DHT 中的定位遵循一个基本原则：每个数据键 k 将被分配给哈希空间中距离该键最近（根据协议定义）的一个或一组对等节点。

1. **数据定位原则：** 在 Chord 中，键 k 存储在其后继节点（successor）上，即环上第一个 $ID \geq k$ 的节点^[4]。在 Kademlia 中，键 k 被存储在距离 k 的 XOR 距离最近的 K 个节点上^[5]。

2. **路由机制：** 节点通过维护一个结构化的路由表来高效地将查询转发给距离目标键越来越近的节点。这种机制类似于一个**跳跃列表 (skip-list)**，使得查找路径的长度被压缩^[1]。
3. **对数级查找：** 无论采用 Chord 的指针表还是 Kademlia 的 k -buckets，其路由算法都能确保在拥有 N 个节点的网络中，查找任何键的平均复杂度可以达到网络规模的对数级别 $O(\log N)$ 。

2.3 节点加入与退出

去中心化网络的动态性要求 DHT 必须能够高效地处理**节点流失 (Churn)**，即节点的持续加入和退出。

1. **节点加入：** 新节点加入系统时，它必须执行一系列初始化步骤：
 - **路由信息初始化：** 新节点需要找到网络中的一个已知节点 w 作为联系，并初始化自身的路由表^[5]。
 - **责任接管：** 新节点会从其相邻节点处接管部分哈希空间和数据键的存储责任^[6]。
 - **通知机制：** 节点加入后必须通知其相邻节点，以便这些节点更新它们指向新节点的指针或前驱/后继关系。
2. **节点退出与容错：** 当节点随机失败或离开时，DHT 依靠容错机制来确保系统的可靠性^[2]：
 - **冗余机制：** 采用冗余存储（如 Kademlia 中将数据存储在 K 个最近节点上）来确保即使主要存储节点失败，数据仍然可被检索^[1]。
 - **周期性维护：** 协议会周期性地运行后台稳定程序（如 Chord 的 `Stabilize()` 和 `Fix\_fingers()`）或刷新机制（如 Kademlia 的 k -bucket PING），以检测故障节点并更新路由表，确保所有指针和后继关系保持最新^[5]。

2.4 容错与负载均衡

容错性和负载均衡是衡量 DHT 实用性的关键指标。

特性	Chord 协议中的体现	Kademlia 协议中的体现
容错性	依赖于周期性的 稳定协议 （如 <code>Stabilize()</code> ）来修复因节点故障导致的环形结构断裂。	依赖于 K 值冗余和 并行查找 。即使路径中 α 个节点失败，并行性也能保证查询继续进行。
负载均衡	节点的存储负载易出现 不均衡 问题。由于节点 ID 在哈希环上的间距不均，某些节点可能负责管理过大的哈希空间，形成负载热点。	具有更好的 统计均衡性 。由于 XOR 距离的随机性和对称性，键的存储位置在统计上均匀分布，有效避免了固有的负载不均衡问题。

为了解决 Chord 的负载不均衡问题，后续的结构化 DHT 协议（如 Pastry）通过调整节点的管理空间划分策略，显著改善了负载的均衡性^[7]。Kademlia 则通过其基于位模式差异的 XOR 距离度量，天生就具有更高的统计鲁棒性和负载均衡性，使其更适用于高流失率的不可信环境^[5]。

3 Chord 协议详解

Chord 协议是由麻省理工学院（MIT）提出的一种开创性的结构化 DHT 协议，它通过构建一个简洁且逻辑性强的环形拓扑结构，实现了高效的、对数级别的路由查找。

3.1 环形拓扑结构

Chord 的设计基础是使用一致性哈希技术将所有节点 ID 和数据键映射到一个 m 位的环形 ID 空间上 [4]。

1. **ID 空间与环：** 整个键空间是一个取值范围从 0 到 $2^m - 1$ 的环，所有的节点 ID 和数据键都被哈希到这个环上的特定位置。
2. **数值顺序度量：** 在 Chord 中，节点间的距离度量是基于模 2^m 的**数值距离和顺序**。
3. **后继节点：** 数据的存储位置遵循一个简单的规则：每个数据键 k 会被分配给环上第一个 ID 大于或等于 k 的节点。该节点被称为 k 的**后继节点（successor）** [8]。

逻辑上，这种结构确保了每个节点都负责管理从其前驱节点 ID 到自身 ID 之间的所有哈希空间。

3.2 一致性哈希

Chord 协议依赖于**一致性哈希（Consistent Hashing）**来分配键和节点，这是其实现可扩展性的关键。一致性哈希确保了当节点加入或退出时，只有邻近节点之间的键需要重新分配，从而最小化了数据迁移的开销 [4]。这种机制保证了 DHT 能够在高度动态的环境中保持高效。

3.3 路由表与指针维护

为了实现快速查找，每个 Chord 节点 n 维护一个被称为**指针表（Finger Table）**的结构化路由信息集合。

1. **指针表的结构：** 指针表包含 m 个条目。第 i 个条目（记作 $finger(i)$ ）记录的是环上 ID 值大于或等于 $n + 2^{i-1} \pmod{2^m}$ 的第一个节点。

2. **后继指针：** 指针表的第一个条目 $finger(1)$ 实际上就是节点 n 在环上的直接后继节点，因此无需单独维护一个后继字段。
3. **维护机制：** Chord 必须周期性地运行**稳定协议（Stabilization Protocol）** 来确保其环形结构和路由信息的准确性。核心的维护操作包括：
 - **Stabilize():** 节点 n 周期性地运行此操作。它询问其后继节点 s 的前驱 p ，并判断如果 p 位于 n 和 s 之间（例如 p 是最近加入的节点），则 n 更新其后继指针指向 p 。
 - **Notify():** 当 `Stabilize()` 检测到新后继时，节点 n 通知其后继节点 s 自己的存在，允许 s 更新其前驱指针指向 n 。
 - **Fix_fingers():** 周期性地运行，更新和修复指针表中的所有 m 个条目，确保所有 2^{i-1} 距离的链接都指向正确的后继节点。

3.4 查询机制

Chord 的路由查询利用其指针表的指数级增长间隔实现快速跳跃。

1. **路由路径：** 当节点 n 查找键 k 时，它会检查其指针表。它找到 ID 距离 k 最近、但 ID 值不超过 k 的最大节点，并将查询转发给该节点。
2. **收敛：** 这种机制使得查询能够沿环形拓扑快速“跳跃”前进，直到查询最终到达存储该键的直接后继节点。
3. **效率：** 在一个拥有 N 个节点的网络中，查找任何键所需的联系节点数量平均为 $O(\log N)$ 。

3.5 节点动态管理

新节点的加入需要进行一系列的初始化和通知操作以融入系统：

1. **初始化：** 新节点 n' 首先需要初始化其前驱和指针表。初始化指针表可以通过执行一系列 `find successor` 查询来完成，最优化的方法可以将初始化时间复杂度降至 $O(\log N)$ 。
2. **接管责任：** 新节点从其直接后继节点处接管它现在负责的键空间。
3. **通知更新：** 新节点通知其后继节点和其他相关节点更新它们的指针和前驱，以纳入 n' 。

然而，研究表明 Chord 的性能对节点加入和离开的频率（流失率）与稳定协议的调用频率之间的平衡非常敏感。同时，纯粹的 Chord 网络中，节点的存储和查询负载被发现是**不均衡的**，容易出现负载热点^[7]。为了解决这种不均衡性，后续的 DHT 改进方案（如 Pastry）通过更细致的键空间划分策略来提高负载的统计均衡性。

4 Kademlia 协议详解

Kademlia 协议是另一种结构化 DHT 协议的典范，其设计哲学着重于在高度动态、不可靠的公网环境中实现卓越的容错性和并行查找能力。与依赖环形顺序的 Chord 不同，Kademlia 采用基于 XOR 运算的距离度量，从而优化了在高流失率下的性能。

4.1 XOR 度量空间

Kademlia 的核心创新在于使用**异或 (XOR)** 运算的结果来定义节点 ID 或数据键之间的“距离”^[5]。

1. **距离计算：** 两个节点或一个节点与一个键之间的距离是通过对它们的二进制 ID 执行 XOR 运算，并将结果视为一个无符号整数来计算的。
2. **距离函数的数学特性：** 之所以选用 XOR 作为距离函数，是因为它满足自反性（节点到自身距离为零）、对称性（从 A 到 B 的距离与从 B 到 A 的距离相同）和三角不等式。
3. **统计临近性：** 密钥和节点 ID 具有相同的格式和长度（例如 IPFS 中为 256 位哈希值）。ID 间的 XOR 距离越短，意味着它们共享的二进制前缀越长，它们在逻辑上就越“近”。这种随机性带来的统计分布使得在地理上遥远的节点如果具有相似的随机 ID，也可以在逻辑上成为“邻居”。

4.2 k-bucket 路由表

Kademlia 的路由表结构由一系列动态的**k-buckets**组成，这些列表共同覆盖了整个 ID 空间。

1. **结构与距离划分：** 路由表被组织成一个逻辑上的二叉树，其叶子节点即为 k-buckets。每个 k-bucket 存储的节点与当前节点 ID 的 XOR 距离范围在一个特定的区间内。例如，第 n 个 k-bucket 包含的节点，其 ID 的第 n 个比特位与本节点 ID 不同，而前 $n - 1$ 个比特位必须相匹配^[5]。
2. **容量 K ：** 每个 k-bucket 中最多可以包含 K 个节点（例如 IPFS 中 $K = 20$ ）。这个 K 值提供了关键的冗余和容错能力，其选择是基于对网络平均流失率的观测而定。

3. **动态更新与分裂：** 路由表中的 k-buckets 是动态分配的。当一个节点发现了一个新的联系并尝试将其插入相应的 k-bucket 时：如果 k-bucket 未满，则直接插入；如果 k-bucket 已满，且其距离覆盖范围包含本节点 ID，则该 bucket 会被分裂成两个新的 k-buckets，将旧内容重新分配到两个新的 bucket 中^[6]。如果一个已满的 k-bucket 的范围不包含本节点 ID，则新节点将被简单丢弃。

4.3 节点查找算法

Kademlia 的查找过程是高度并行化的迭代过程，旨在快速找到距离目标键最近的 K 个节点。

1. **迭代查询：** 查找由发起节点开始，该节点首先从其 k-buckets 中选出距离目标键最近的 α 个节点，并向它们并行地发送 `FIND_NODE` 或 `FIND_VALUE` 请求^[5]。
2. **结果返回：** 接收请求的节点会在其自己的 k-buckets 中查找并返回它们所知道的距离目标键最近的 K 个节点。
3. **收敛：** 发起请求的节点会迭代地向新收到的 K 个最佳节点发出请求。由于 XOR 距离的特性，每一步迭代都会确保找到的节点越来越接近目标键。
4. **停止条件：** 迭代持续进行，直到没有返回比当前最佳结果更接近目标键的节点为止。最终结果列表中最好的 K 个节点即为整个网络中距离目标键最近的节点。

4.4 并行查找优化

Kademlia 的设计通过引入并行度参数 α 来显著优化查询延迟。通过同时向多个节点发起查询，即使路径中有节点失败或响应缓慢，查询也可以通过其他并行节点继续推进。这种设计牺牲了查找路径的理论最短性，换取了**低延迟**和对**节点失败的高容忍度**，使其非常适合在高流失率的公网环境中运行。

4.5 数据复制与缓存

Kademlia 协议通过 `STORE` 和 `FIND_VALUE` 消息管理数据的存储和检索。

1. **数据存储：** 为了存储一个 (键, 值) 对，节点首先使用 `FIND_NODE` 找到距离该键最近的 K 个节点，然后向这些节点发送 `STORE` 消息。这种冗余存储确保了数据的高可用性和容错能力。

2. **数据检索：** 检索数据时使用 `FIND_VALUE` 消息，流程与节点查找相似。如果接收方存有请求的键，它将直接返回相应的值；否则，它返回已知的 K 个最近节点，继续迭代查找。
3. **稳定性机制：** Kademlia 倾向于将长时间保持连接的节点存储在 k-buckets 中，以提高网络的稳定性。当一个 k-bucket 满了并发现了一个新节点时，系统会 PING k-bucket 中**最近最少见**的节点。如果老节点存活，新节点会被放置在一个**替换缓存 (replacement cache)** 中，只有当 k-bucket 中原有的节点停止响应时，替换缓存中的节点才会被用来替换死节点。此外，节点会周期性地刷新（例如每小时）任何在过去一小时内未进行过节点查找的 k-buckets，以确保路由表的活跃度^[6]。

5 Chord 与 Kademlia 的比较

Chord 和 Kademlia 是结构化 DHT 领域的两种代表性协议，它们都成功地实现了高可扩展性和对数级查找效率 $O(\log N)$ 的目标。然而，由于它们采用了截然不同的底层机制——Chord 基于**数值顺序**和**环形拓扑**，而 Kademlia 基于**XOR 距离**和**二进制树分区**——使得两者在实际性能、容错性和实现复杂度上存在显著差异。

5.1 路由效率对比

虽然两种协议在理论上都达到了 $O(\log N)$ 的平均查找复杂度，但 Kademlia 在实际网络中的表现通常更具优势。

Chord 的路由是**确定性跳跃**，沿着环形结构精确地转发查询到下一个最接近目标的节点^[9]。这种确定性使其路径计算理论上效率很高，但必须依赖于精确且持续维护的指针表。相比之下，Kademlia 的查找是**迭代式并行查询**。通过同时向 α 个（例如 $\alpha = 3$ ）节点发起请求，Kademlia 牺牲了查找路径的理论最短性，换取了低延迟和对节点失败的高容忍度^[5]。在高延迟或高流失率的网络中，Kademlia 的并行性能够有效避免因等待单个节点响应而导致的性能瓶颈，因此在实际应用中往往表现出更快的查询速度。

5.2 可扩展性分析

两者都具备出色的可扩展性，能够高效地扩展到数百万个节点^[2]。然而，它们在处理节点加入和退出（流失率）时表现出不同的特性：

对比维度	Chord 协议（基于数值顺序）	Kademlia 协议（基于 XOR 距离）
路由拓扑	逻辑环形结构，强调绝对顺序。	基于 XOR 距离的二叉树分区结构，强调位模式相似性。
负载均衡性	易出现负载不均衡（Load Imbalance）。由于节点 ID 在环上间距的随机性，某些节点可能负责管理过大的哈希空间，成为 负载热点 。	具有更好的 统计均衡性 。XOR 距离的随机性和对称性使得键的存储位置在统计上趋于均匀分布，有效避免了固有的负载不均衡。
对流失率（Churn）的适应性	敏感。高流失率要求高频率地运行 <code>Stabilize()</code> 和 <code>Fix_fingers()</code> 维护协议，增加了网络开销。	适应性强。通过 K 冗余和替换缓存机制，优先选择长时间连接的稳定节点，增强了网络的弹性。

5.3 容错性能对比

Kademlia 在设计上比 Chord 更具容错性，尤其适用于高度动态和不可信的公网环境。

- 1. **冗余机制：** Kademlia 通过将数据存储在距离键最近的 K 个节点上，天然地提供了数据冗余和多路径选择。在 IPFS 中 $K = 20$ ，这种冗余确保了对于 ID 空间的每个距离范围，系统都会保留多达 20 个链接，即使部分节点失败，仍有备选路径。相比之下，Chord 的容错主要依赖于其严格的稳定协议来修复断裂的环结构。
- 2. **故障检测与替换：** Kademlia 通过 PING 机制和**替换缓存**来管理路由表中的节点活跃度。只有在 k-bucket 中的节点被 PING 证明失败后，系统才会使用替换缓存中的新节点进行替换。这种机制确保了路由表主要由已证明稳定的节点构成，提高了网络的可靠性。

5.4 实现复杂度

Chord 的基础环形结构和一致性哈希的概念相对简洁^[4]。然而，为了应对高流失率和潜在的负载不均衡问题，Chord 必须依赖于复杂的**稳定协议**（`Stabilize()`，`Notify()`，`Fix\_fingers()`）来持续修复和维护拓扑结构的一致性和精确性^[9]。

Kademlia 的核心路由表机制——k-buckets 的动态分裂和替换缓存策略——在实现上具有一定的复杂性^[6]。然而，一旦实现，其基于 XOR 距离的查找机制和并行性使其在应对不可预测的公网流失和延迟时，具有更强的工程实用性。这使得 Kademlia 成为现代大规模 P2P 系统（如 BitTorrent Mainline DHT 和 IPFS）的首选协议。

6 总结与展望

分布式哈希表技术作为去中心化网络基础设施的核心组成部分，通过 Chord 和 Kademlia 这两种代表性协议的发展，为大规模、高动态性的 P2P 系统提供了高效、可靠的键值存储和查找方案。Chord 和 Kademlia 协议均成功实现了 DHT 的三大核心目标：去中心化、容错性和对数级可扩展性。

不过尽管 Chord 和 Kademlia 已经取得了巨大的成功，并被广泛应用于 BitTorrent 和 IPFS 等主流系统，但 DHT 技术仍在不断演进，以适应更复杂的网络环境 and 应用需求。

现代 DHT 的实践证明，理论协议必须与复杂的网络环境深度融合。例如，IPFS 对 Kademlia 进行了多项关键定制，包括： K 值优化，网络连通性处理和双 DHT 架构。

未来研究方向包括：

1. **负载均衡的持续优化：** 需要探索更精细的键空间划分和节点职责分配策略，进一步解决结构化 DHT 协议（如 Chord）固有的负载不均衡问题。
2. **安全性和抗攻击性：** 增强 DHT 协议对恶意攻击（例如女巫攻击，Sybil Attacks）的抵抗力，这可能涉及更先进的节点身份验证和路由表信任机制。
3. **结合网络拓扑的路由优化：** 当前 DHT 路由主要基于 ID 距离，未来的研究应探索如何结合节点的实际地理位置或网络延迟数据来优化路由表的构建，从而实现更低的实际查询延迟，进一步提升用户体验。

综上，分布式哈希表技术是去中心化网络基础设施的关键支柱。通过对 Chord 和 Kademlia 协议的深入理解和不断优化，DHT 技术将持续在网络存储、内容分发和下一代互联网（Web3）应用中发挥核心作用。

参考文献:

- [1] IPFS. "DHT 基础概念." Accessed: 2025-11-10. [Online]. Available: <https://docs.ipfs.tech/concepts/dht/>
- [2] Wikipedia. "Distributed hash table." Accessed: 2025-11-10. [Online]. Available: https://en.wikipedia.org/wiki/Distributed_hash_table
- [3] IPFS. "DHT 基础概念." Accessed: 2025-11-10. [Online]. Available: <https://docs.ipfs.tech/concepts/dht/>
- [4] Wikipedia. "Chord (peer-to-peer)." Accessed: 2025-11-10. [Online]. Available: [https://en.wikipedia.org/wiki/Chord_\(peer-to-peer\)](https://en.wikipedia.org/wiki/Chord_(peer-to-peer))
- [5] Wikipedia. "Kademlia." Accessed: 2025-11-10. [Online]. Available: <https://en.wikipedia.org/wiki/Kademlia>
- [6] P. Maymounkov and D. Mazières. "Kademlia: A Peer-to-peer Information System Based on the XOR Metric." Accessed: 2025-11-10. [Online]. Available: <https://pdos.csail.mit.edu/~petar/papers/maymounkov-kademlia-lncs.pdf>
- [7] J. Suese. "Chord网络中节点负载是不均衡的；Pastry由于采用了节点间距的一半作为节点的管理空间，使得节点负载的均衡性有很大改善." Accessed: 2025-11-10. [Online]. Available: <https://jsuese.scu.edu.cn/zh/article/43955374/>
- [8] M. F. Kaashoek and D. Karger. "Chord: A Distributed Lookup Protocol for Peer-to-Peer Networks." Accessed: 2025-11-10. [Online]. Available: https://pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf
- [9] M. F. Kaashoek and D. Karger. "Chord: A Distributed Lookup Protocol for Peer-to-Peer Networks." Accessed: 2025-11-10. [Online]. Available: https://pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf